

Homework 2: MapReduce, Terraform, Docker... and Claude Code!

New Attempt

- Due 26 Jan by 12:00
- Points 10
- Submitting a text entry box, a website url, or a file upload

Part I: Reading and Concepts!

You will be reading about Map Reduce to learn how you can distribute large workloads across different machines. Unlike the last week, this is a research paper, so you might want to take a look at [How to read a paper \(https://web.stanford.edu/class/ee384m/Handouts/HowtoReadPaper.pdf\)](https://web.stanford.edu/class/ee384m/Handouts/HowtoReadPaper.pdf)

This time, please your review of the [Map Reduce paper \(https://dl.acm.org/doi/10.1145/1327452.1327492\)](https://dl.acm.org/doi/10.1145/1327452.1327492) on Piazza, noting (1) what you liked about it, (2) what you didn't like about it. Be ready to show this review to your TA during your mock interview this week.

Part II: Get Started With Terraform

Overview

In this assignment, you will learn how to automate the same EC2 setup process you went through in Homework 1, but instead use Infrastructure as Code with *Terraform*. Start by making sure you don't have an instance running by checking your AWS console. Instead of all the menu/button clicking in Homework 1, here you will use Terraform to *magically* start up an instance!

One key concept here is the *reproducibility* of the workflow involved. We will see more of this later, but for now, let's just get an idea of what "Infrastructure as Code (IaC)" looks like.

Automate with Terraform

You can set up a new EC2 instance the same way you did in week 1, but you'd need to repeat the same process by hand every time. For those of you who are not fond of clicking through boxes manually to manage your cloud resources, fear not! We will introduce you to [Terraform \(https://developer.hashicorp.com/terraform\)](https://developer.hashicorp.com/terraform).

So, let's look at the same setup with code using Terraform. First, you will need to install [Terraform \(https://developer.hashicorp.com/terraform/install\)](https://developer.hashicorp.com/terraform/install) and [AWS CLI \(https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html\)](https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html).

Setup Instructions

1. Create a new directory
2. Download `main.tf` (<https://northeastern.instructure.com/courses/239676/files/39404445?wrap=1>) 
(https://northeastern.instructure.com/courses/239676/files/39404445/download?download_frd=1)
(https://northeastern.instructure.com/courses/222926/files/36008606/download?download_frd=1)

and `terraform.tfvars` (<https://northeastern.instructure.com/courses/239676/files/39404444?wrap=1>) ↓
(https://northeastern.instructure.com/courses/239676/files/39404444/download?download_frd=1)
(https://northeastern.instructure.com/courses/222926/files/36008607/download?download_frd=1) files into your new folder

3. Read through `main.tf` to start to understand how it translates manual "clicking" in the AWS console into code. Try copying the file into Claude in learning mode!

AWS Credentials

On the AWS Academy Learner Lab page, click start lab. Once the lab has started (button is green), click on AWS details. You will find your credentials AWS CLI here.

To configure credentials on your local machine, open up a terminal and enter `aws configure`, enter your access key id, secret access key. Then, enter `aws configure set aws_session_token <YOUR SESSION TOKEN>`. Verify your credentials with `aws sts get-caller-identity`.

Configure terraform.tfvars

Fill out the values in this `terraform.tfvars` file. This file stores your important data and should be added to `.gitignore` if it's in a git repo.

For obtaining the values in `terraform.tfvars`:

- The `ssh_cidr` is your **IPv4 address** (<https://whatismyipaddress.com/>) plus `/32`, for example: `"192.168.0.1/32"`
- The `ssh_key_name` should be the name of an existing AWS key pair that you created in the AWS Console
- Keep these values in quotes

Use Existing AWS Key Pair

Use an existing AWS key pair that you created previously from earlier assignment. Make sure you have:

1. **AWS Key Pair Name:** The name of your existing key pair in AWS (use this for `ssh_key_name` in `terraform.tfvars`)
2. **Private Key File:** The downloaded .pem file from when you created the key pair (you'll need the path to this file for the SSH connection later)

Deploy the Infrastructure

Now we're ready to go! Run:

```
terraform init
terraform apply -auto-approve
```

If everything goes well, you will see your new EC2 instance's public DNS in the console! If it does not go well, try interacting with Claude in learning mode with your error messages!

Connect via SSH

Replace `<YOUR-EC2-PUBLIC-DNS>` with the previous output and `<PATH-TO-YOUR-PEM-KEY>` with the path to your private key file, then enter:

```
ssh -i <PATH-TO-YOUR-PEM-KEY> ec2-user@<YOUR-EC2-PUBLIC-DNS>
```

For example:

```
ssh -i ~/.ssh/my-key.pem ec2-user@ec2-xx-xx-xx-xx.us-west-2.compute.amazonaws.com
```

You should see the SSH session successfully created.

☐	terraform-crea...	i-0bd20694252d3d898	 Running  	t2.micro	 2/2 checks passed View alarms +
--------------------------	-------------------	---------------------	---	----------	---

Navigate to your AWS web console in your region and you can see your EC2 and security groups. Your dashboard should show the new instance running!

Clean Up

When you are done, always destroy resources to manage your budget:

```
terraform destroy -auto-approve
```

Head back to your dashboard to confirm this worked!

☐	terraform-crea...	i-0bd20694252d3d898	 Terminated  	t2.micro	-	View alarms +	us-west-2a
--------------------------	-------------------	---------------------	--	----------	---	-------------------------------	------------

Result

Terraform makes infrastructure management WAY less painful than it was in Homework 1. By using existing AWS key pairs, we avoid the complexity of managing SSH key generation and focus on the infrastructure deployment itself. We encourage you to read the official Terraform tutorials on AWS. Make sure you can explain what is in the main.tf file in your mock interview.

Note: Your EC2 instance will be assigned a different IP address every time it starts from a stopped, terminated, or restarted state. See how Elastic IP addresses can help (at a cost).

Part III: Getting Started with Lightweight Containers and Docker

Overview

Well, you now have started up virtual machines on TWO different cloud providers (Google Cloud Platform in Homework 1 and AWS now with Terraform, above!).

Let's get the simple Go program from Homework 1 over to AWS (or even Azure, or the compute cluster at Northeastern!). How do you *easily* do that? You might say you can simply clone from your repo, but what if the other machine doesn't have Go? What if your program runs in Windows but the other machine you want to run it on is running Linux or MacOS?

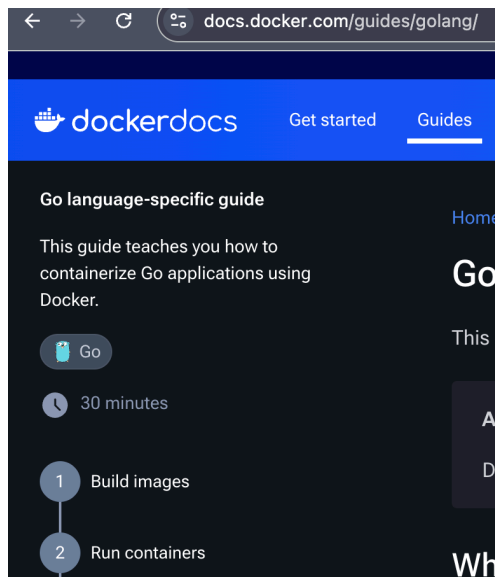
Docker (<https://www.docker.com/>) is one way to address this issue. In this assignment, you will learn about how to "contain" and run your code in a Docker container, and deploy the entire container to another machine.

Contain Your Program!

First change the line that runs your code on `localhost` to one that will run it on a server, by making the following change to your simple **Go web service** (<https://go.dev/doc/tutorial/web-service-gin>) from Homework 1:

`router.Run("localhost:8080")` to `router.Run(":8080")`

Now, let's try running it a *container*. Install **Docker Desktop** (<https://docs.docker.com/get-started/get-docker/>). Refer to **the first two modules** (<https://docs.docker.com/guides/golang/>) from the Docker Go tutorials (Build Images, and Run Containers, shown below).



This is a simpler *main.go* than you did in Homework 1, but similar! Replace it with your own *main.go* from Homework 1, and verify that the message `[GIN-debug] Listening and serving HTTP on :8080` is printed at the end.

Then, open another terminal and send a GET request just as you did in Homework 1. You should see the album data displayed in the terminal!

Let's save your code *and* the Dockerfile to a Git repository. Note that you should leave large files like zip/binaries out of your repo. Read more on **manage and store large files** (<https://learn.microsoft.com/en-us/azure/devops/repos/git/manage-large-files?view=azure-devops>) if this is all quite new to you.

Now, we can host our simple server in the AWS cloud!

Move to the Cloud (again!)

Set up a new EC2 instance with Terraform, as you did earlier. Install Git and Docker within this image. Clone your Git repo, and run your containers just like you did on your local machine, but this time it is in the cloud! Once your container is successfully up and running, try sending a GET request to the public IPv4 address of this EC2 instance instead of localhost. You should see albums printed!

More on Docker

Explore additional Docker guides from the **official resources** (<https://docs.docker.com/get-started/>). In particular, follow the *Introduction*, the basics and building images in *Docker Concepts*, and *Docker Workshop*.

Result

CONGRATS! Docker is an industry standard way to move code over to a server, and now we can go back to our exciting backend for the album store, drop the localhost and actually hit this from a client on the outside! BUT this introduces issues of latency, opportunities for failure, and ACTUALLY doesn't even scale very well!!!

You now have contained your program in an EC2 instance that did not have Go installed. Why don't you need to install Go? Can you confidently walk someone through what happens in the Dockerfile?

Things you can explore:

- What are pros and cons of using containers? What is the difference if we run the go server directly outside of container in the EC2 instance?
- You have installed Git and Docker on this instance, great. But what if for some reason the machine breaks or if you want to repeat the same process on a different machine? You'd have to manually install everything again. That's no fun. Check out [AMI](https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AMIs.html) (<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AMIs.html>) and [how it differs from Docker image](https://stackoverflow.com/questions/41686811/difference-between-docker-and-ami) (<https://stackoverflow.com/questions/41686811/difference-between-docker-and-ami>).

Part IV: Check this out!

Now that it is SO easy to create instances, let's create 2! Set them up ... fill in the IP addresses with your instances, and try this script! What happens and why?

Be ready to discuss this in your interview :)

```
import requests

# Correct IP addresses for both instances
EC2_URL1 = "<ONE OF YOUR ADDRESSES>:8080/albums"
EC2_URL2 = "<THE OTHER OF YOUR ADDRESSES>:8080/albums"

POST_DATA = {
    "id": "4",
    "title": "The Modern Sound of Betty Carter",
    "artist": "Betty Carter",
    "price": 49.99
}
HEADERS = {
    "Content-Type": "application/json"
}

def data_from_both(url1, url2):
    try:
        response = requests.get(url1)
        print(f"Instance 1 response: {response.text}")
    except requests.exceptions.RequestException as e:
        print(f"Error occurred while testing {url1}: {e}")

    print("\n\nand...")
    try:
        response = requests.get(url2)
        print(f"Instance 2 response: {response.text}")
    except requests.exceptions.RequestException as e:
        print(f"Error occurred while testing {url2}: {e}")
    return

print("Starting data test...")

data_from_both(EC2_URL1, EC2_URL2)
```

```
print("\n\nand adding...")
try:
    response = requests.post(EC2_URL2, json=POST_DATA, headers=HEADERS)
    print(f"Instance 2 response: {response.text}")
except requests.exceptions.RequestException as e:
    print(f"Error occurred while testing {EC2_URL2}: {e}")

data_from_both(EC2_URL1, EC2_URL2)

print("uhoh... what happened?")
```

Part V: An Exploration of a System using Claude Code!

Welcome to your *Innovation Sandbox*! A video to help you get things set up is here!

<https://youtu.be/m5oCcXARzNg> ➞ <https://youtu.be/m5oCcXARzNg>



<https://youtu.be/m5oCcXARzNg>

(Thank you Reed!). Note that your username is your email address prefix. For example, *huang.ruid* for huang.ruid@northeastern.edu (<mailto:huang.ruid@northeastern.edu>). Enter your NU email prefix here: <https://d38k6e0c5ds6dp.cloudfront.net/> ➞ <https://d38k6e0c5ds6dp.cloudfront.net/> to start the MFA process!

As in the video, you will first need to request a lease and shortly after that, you will get an email that includes your SSH key!

Once you have set up your private key on our own machine, the next step is to *ssh* into your new world (called an **Innovation Sandbox** ➞ <https://aws.amazon.com/solutions/implementations/innovation-sandbox-on-aws/>) to play in! 😊 But guess what? This time you are **NOT** writing code! You are analyzing it with the help of **Claude Code** ➞ <https://code.claude.com/docs/en/overview>. The goal is to see if Claude Code can help you understand the Makefile, code, and logs from an application that has a bit of a crazy bug!

This is NOT supposed to take a lot of your time. See if, after you have watched the video for how to interact with the mystery application and Claude Code, you can accurately pinpoint the bug with evidence (from Claude Code!), in **less than 1 hour**!

Details (as shown in the video!)

Specifically, you have one “*mystery*” application you can run, and your job is to use Claude Code to help you analyze what is happening when you run the mystery application. Your challenge is to see if you can get Claude Code to tell you what is going on, not just by asking it about the code base, but also the logs from the GIN Go server and even **CloudWatch** ➞ <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-cloudwatch.html> logs from EC2.

Claude code is pre-installed, enter 'claude' in the terminal to enter Claude Code.

Navigate to ~/mystery, run 'make deploy ' and you will see a *Lambda URL*. But wait... what is a **Lambda URL** ➞ <https://docs.aws.amazon.com/lambda/latest/dg/urls-configuration.html>? Ask Claude Code to give you some ideas!

The `/album-count/:album_id` endpoint will modify the count of the album with that id by 10,000. Try sending a few requests with `curl -X POST <lambda_url>/album-count/1`.

The whole point of this task is to ask Claude Code to help you analyze this system! Ask it to search the Lambda's logs, read the code, etc. Have fun with this!

WHAT TO HAND IN, WHERE AND WHEN!

Part I, your review: Please post your thoughts on **Piazza** by **Wed Jan 21 3pm Pacific Time**. You will be showing this in your mock interview next week! Note there are no wrong answers here :)

Part II, Terraform: Please submit your `main.tf`

[_ \(https://northeastern.instructure.com/courses/222926/files/36008606?wrap=1\)](https://northeastern.instructure.com/courses/222926/files/36008606?wrap=1)

[_ \(https://northeastern.instructure.com/courses/222926/files/36008606/download?download_frd=1\)](https://northeastern.instructure.com/courses/222926/files/36008606/download?download_frd=1)

and `terraform.tfvars` [_ \(https://northeastern.instructure.com/courses/222926/files/36008607?wrap=1\)](https://northeastern.instructure.com/courses/222926/files/36008607?wrap=1)

[_ \(https://northeastern.instructure.com/courses/222926/files/36008607/download?download_frd=1\)](https://northeastern.instructure.com/courses/222926/files/36008607/download?download_frd=1) files, along with screenshots of your victory, on **Canvas** by **Monday Jan 26 9am Pacific Time**, and make sure your **Teaching Team member is set up to appreciate your beautiful code and assignment organization on your Github repo!**

Part III, Docker: Please submit *more* screenshots of your victory on **Canvas** by **Monday Jan 26 9am Pacific Time**.

Part IV, What's going on?! Please submit final screenshots showing the results of what you are seeing, and comment on what is going on in a text entry on **Canvas** by **Monday Jan 26 9am Pacific Time**.

Part V, Claude Code! Please keep track of your time for this portion of the assignment. Upload a short report **Canvas** by **Monday Jan 26 9am** that includes:

1. The amount of time you spent (outside of understanding the video) interacting with Claude Code asking questions. See if you can get the answers you need in under 1 hour!? (it is ok if you don't!)
2. A description of the *mystery bug* in terms of what evidence you were able to pull out of system logs with Claude's help. Feel free to copy/paste the relevant portions of the system logs, and please indicate which log file they are pulled from.
3. Your `.jsonl` file (as demonstrated at the end of the video), showing how you used Claude Code to analyze the application's behaviour. Feel free to upload the `.jsonl` file as is to Canvas as "raw data" :)

Mock Interview Scoring! (1)

Criteria	Ratings						Pts				
Code Quality	2 Pts Full Marks Excellent! Very clear and concise, outstanding documentation, readability, and maintainability. Beautiful code!		1.5 Pts Good quality Clarity could be improved, in particular documentation.		1 Pts Could use improvement Code readability and maintainability could be improved, this will come with practice!		0.5 Pts Major improvement expected Code quality is poor, not something that could easily be shared with a team in a code walk.		0 Pts No code submitted by due date.		2 pts
Code Completion	2 Pts Full Marks Excellent! Code is complete and runs without error, thought has been put into testing and error checking.		1.5 Pts Small bugs Minor runtime errors, cleared up in interview time, but otherwise code runs correctly.		1 Pts Needs improvement Code does not run and suffers from major errors, and requires further testing.		0.5 Pts Needs major improvement Some code was submitted, but has significant errors and has not been subjected to adequate testing.		0 Pts No Code was submitted by due date.		2 pts
Code understanding	2 Pts Full Marks Excellent job in answering questions about the code and analysis posed by both the TA and other group members!		1.5 Pts Good answers Answers to questions were clear, but there is room to improve and go deeper into technical explanations.		1 Pts Communication improvements expected Ability to show a deeper understanding is expected.		0.5 Pts Major Communication Issue Answering questions will require more practice, both in terms of listening carefully to the question that is asked, and thinking deeply about the tradeoffs in the answers.		0 Pts No Code was submitted by due date.		2 pts
Listening and Engaging with team	2 Pts Full Marks Excellent job of contributing to the group		1.5 Pts Strong interaction Some elements of group interaction can be improved, in particular making		1 Pts Some improvements expected Ensure listening is uninterrupted and		0.5 Pts Major Engagement Issue Did not appear to be		0 Pts No questions asked No interaction		2 pts

Criteria	Ratings					Pts
	discussion! Listening closely to others, asking insightful questions, helping consider alternatives!	questions/comments specific and insightful.	the discussion is progressing. Do not repeat questions/comments that have already been shared.	listening, and was unable to contribute beyond superficial comments.	with team members	
Understanding of Concepts and Tradeoffs	2 Pts Full Marks Excellent job of mapping concrete code examples to high level concepts! This was demonstrated throughout the interview, and made abundantly clear in the posting on the reading on Piazza!	1.5 Pts Good sense of tradeoffs Is able to see the connection to higher level concepts, and is working on communicating the relationships between code examples and concepts. Posting on Piazza was strong.	1 Pts Room for improvement With practice, the ability to reason about concepts and tradeoffs and communicate clear points will be improved. Developing this new skill, posting on Piazza showed promise.	0.5 Pts Needs Major improvement Posting on Piazza needs more depth. Currently not connecting examples to concepts and tradeoffs, but aware of the nature of this relationship.	0 Pts No Marks No posting on the reading on Piazza.	2 pts
Total points: 10						